

Algorithms & Data Structures

Industry Scenario Problems – Foundation Practice Set (Days 1 to 5)

Module: Algorithms & Data Structures using Java · Batch: PGCP-AC August 2026

General Instructions

Dear Students,

This practice set is designed to help you build a strong foundation in the topics covered from Day 1 to Day 5. Every problem is based on a real-world software situation, the same kind of problem you will meet in interviews and on the job. The difference is that each problem here comes with extra guidance, so that you can understand **what** to build and **how** to build it.

The problems are arranged in increasing order of difficulty, from **Level 1** to **Level 10**. Please attempt them **in order**. Each one uses an idea from the previous ones, so skipping ahead will make the later problems harder than they need to be.

Each problem contains the following help sections:

Section	How to use it
Concept Refresher	A short reminder of the data structure or idea used. Read this first.
Example	Exact input and output. Your program must produce this output.
Starter Code	Class and method names to begin with. Fill in every // TODO yourself.
Step-by-Step Guide	The logic broken into small steps. Use it when you don't know where to begin.
Common Mistakes	Errors students usually make in this problem. Check your code against them.
Reference	Where to revise the concept: the course portal and visual tools.

Students are expected to follow these steps for every problem:

1. **Understand** – Write the problem in your own words. Note the input, the output and the operations required.
2. **Draw** – Draw the data structure on paper and trace the example by hand before writing any code.
3. **Algorithm and Pseudocode** – Write the steps of each operation in simple English, then as pseudocode.
4. **Java Implementation** – Complete the starter code. Compile and run after finishing *each* method, not only at the end.
5. **Complexity** – Fill in the complexity table given with the problem.
6. **Test** – Run the given example, the listed edge cases and at least two inputs of your own.

Use of AI Tools

You are not permitted to ask an AI tool to write the code for these problems. If you are stuck, read the Step-by-Step Guide again, revise the Reference, or discuss the *idea* with a classmate. Please do not share code.

You may use an AI tool only to **explain a concept** you did not understand (for example, “explain how a linked list node stores the next address”), or to review code you have already written yourself.

Problem Map

Level	Scenario	Concept Practised	Day
1	Exam eligibility from attendance	Array traversal, counting, streaks	1, 2
2	Railway berth booking	Array as a fixed set of slots	2
3	Online shopping cart	ArrayList of objects	2
4	Crash report (stack trace)	Stack using an array	2
5	Office printer queue	Queue using a linked list	2, 3
6	Train coach arrangement	Singly linked list operations	3
7	Hospital waiting list	Doubly linked list	3
8	Banking utilities	Basic recursion	4
9	Payment kiosk coin combinations	Recursion tree, repeated work	4
10	Library book catalogue	Binary search tree and traversals	5

Question 1 - Exam Eligibility from Attendance

Level 1 of 10 · Topic: Arrays, traversal, counting · Day 1, 2 · Beginner

Scenario

A college attendance system stores a student's attendance as an array: 1 means present and 0 means absent, one value per working day. The exam cell needs a report for each student:

1. Number of days present, and the attendance percentage.
2. Whether the student is **eligible** for the exam (attendance must be at least 75%).
3. The **longest run of continuous presence** (used for a "regular student" badge), and the longest run of absence.
4. If the student is not eligible: how many more days they must attend continuously to reach 75%.

Concept Refresher

An array stores values in continuous positions, and we visit them one by one with a `for` loop. To find a **streak**, keep two variables: the *current* streak and the *best* streak so far.

Example

Input: attendance = [1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0] (15 days)

Output: Days present : 11 of 15
Attendance : 73.33 %
Eligible : NO
Longest presence : 5 days (day 10 to day 14)
Longest absence : 2 days (day 8 to day 9)
Days needed for 75%: 1 (12 of 16 = 75.00 %)

Conditions

- Do not use any built-in method to count. Traverse the array yourself.
- Print the percentage with two decimal places (`String.format("%.2f", value)`).
- Days are numbered from 1 in the output, though array indexes start at 0.

Starter Code

```
public class AttendanceReport {

    static int countPresent(int[] att) {
        // TODO: count how many 1s are in the array
        return 0;
    }

    static double percentage(int present, int total) {
        // TODO: careful - present / total with int values gives 0!
        return 0.0;
    }

    static void longestStreak(int[] att, int value) {
        // TODO: find the longest continuous run of 'value' (1 or 0)
        //          print its length and its start and end day
    }

    static int daysNeeded(int present, int total) {
        // TODO: keep adding one present day until percentage >= 75
        return 0;
    }

    public static void main(String[] args) {
        int[] att = {1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0};
        // TODO: call the methods above and print the report
    }
}
```

Step-by-Step Guide (for the longest streak)

1. Create `current = 0`, `best = 0` and `bestEnd = -1`.
2. Visit each index `i`. If `att[i] == value`, increase `current` by 1. Otherwise, reset `current` to 0.
3. After updating `current`, if `current > best`, set `best = current` and `bestEnd = i`.
4. After the loop, the streak ends at index `bestEnd` and starts at index `bestEnd - best + 1`. Add 1 to each for day numbers.

Your Tasks

1. Trace the longest-presence logic by hand for the example, writing `i`, `current` and `best` in a table.
2. Complete all the methods and match the example output exactly.
3. Fill in this table:

Operation	Time Complexity	Reason
Count present days	?	?
Longest streak	?	?
Days needed for 75%	?	?

Edge Cases to Consider

- All present (100%)
- All absent (the longest presence is 0)
- Exactly 75% (eligible or not?)
- An empty array

Common Mistakes

Writing `present / total * 100` with `int` values. This prints 0. Convert one value to `double` first.
Forgetting to check the streak once more when the array *ends* during a streak (in the example, the best streak ends at day 14).

Think Before Coding

Can you find the present count, the longest presence and the longest absence in a **single** loop?

Reference

Course portal, Day 1 (algorithm constructs) and Day 2 (arrays).

On the Job

HR attendance systems, gym check-in streaks, and “days active in a row” badges in apps like Duolingo and LeetCode.

Question 2 - Railway Berth Booking

Level 2 of 10 · Topic: Arrays as slots, searching with conditions · Day 2 · Beginner

Scenario

A railway reservation system manages the berths of a sleeper coach. A passenger asks for a berth type (for example, Lower). The system gives the **first free berth of that type**. If none is free, it gives the **first free berth of any type**. Passengers can also cancel bookings.

In an Indian Railways sleeper coach, the berth type depends on the berth number:

seatNo % 8	1 or 4	2 or 5	3 or 6	7	0
Berth type	Lower (LB)	Middle (MB)	Upper (UB)	Side Lower (SL)	Side Upper (SU)

Concept Refresher

Use a `boolean[] booked` array where `booked[s]` is `true` if berth `s` is taken. Make the array size `N + 1` and ignore index 0, so that berth numbers match array indexes directly.

Example

Coach with 16 berths. Already booked: 1, 2, 4, 7

```
book("LB")    -> Berth 9 (LB)           (LB berths are 1, 4, 9, 12)
book("SL")    -> Berth 15 (SL)          (SL berths are 7, 15)
book("SL")    -> No SL free. Berth 3 (UB) allotted instead
cancel(4)     -> Berth 4 cancelled
book("LB")    -> Berth 4 (LB)
available()   -> 9 berths free
```

Conditions

- Berths are numbered from 1 to N.
- Cancelling a berth that isn't booked, or a number outside 1..N, must print an error message instead of crashing.
- When the coach is full, `book` must report “Waiting List”.

Starter Code

```
public class SleeperCoach {
    private boolean[] booked;
    private int n;

    public SleeperCoach(int n) { /* TODO: create array of size n + 1 */ }

    static String berthType(int seatNo) {
        // TODO: use seatNo % 8 and the table above
        return "";
    }

    public int book(String preferred) {
        // TODO: pass 1 - first free berth of the preferred type
        // TODO: pass 2 - first free berth of any type
        // TODO: return -1 if the coach is full
        return -1;
    }

    public boolean cancel(int seatNo) { /* TODO */ return false; }
    public int available() { /* TODO */ return 0; }
    public void printChart() { /* TODO: e.g. 1:LB:X 2:MB:X 3:UB:_ ... */ }
}
```

Step-by-Step Guide (for book)

1. Loop s from 1 to n . If berth s is free **and** `berthType(s)` equals the preferred type, mark it booked and return s .
2. If the first loop found nothing, loop again from 1 to n and take the first free berth of any type.
3. If the second loop also finds nothing, return -1.

Your Tasks

1. Write `berthType` first and print the type of every berth from 1 to 16. Check it against the table.
2. Complete the class and reproduce the example exactly.
3. Add `availableByType()`, which prints how many berths of each type are free.
4. State the time complexity of `book`, `cancel` and `available`. Which one could be made $O(1)$ by keeping an extra counter?

Edge Cases to Consider

- A full coach
- Cancelling twice
- Berth number 0 or $N + 1$
- Preferred type spelled in lower case ("lb")

Common Mistakes

Comparing strings with `==` instead of `.equals()`.

Making the array of size N and then using `booked[N]`, which throws `ArrayIndexOutOfBoundsException`.

Think Before Coding

Why does the system loop twice instead of once? Could a single loop remember the first free berth of any type *while* searching for the preferred one?

Reference

Course portal, Day 2 (arrays). Revise array indexing and boundary checks.

On the Job

Seat selection in IRCTC, BookMyShow and airline check-in all work on the same idea: a fixed set of slots, each free or taken.

Question 3 - Online Shopping Cart

Level 3 of 10 · Topic: ArrayList of objects · Day 2 · Beginner

Scenario

An online stationery store needs the cart module for its website. Each cart line has a product name, a unit price and a quantity. The cart must support adding, removing and updating items, and must compute the bill using the store's offer rules:

- **10% discount** if the subtotal is Rs. 1000 or more (use integer division).
- **Rs. 40 delivery charge** if the amount after discount is below Rs. 500.

Concept Refresher

An `ArrayList<CartItem>` stores objects in order and grows automatically. Use `get(i)`, `size()`, `add(obj)` and `remove(i)`. Each `CartItem` is a small class with fields such as name, price and qty.

Example

```
add("Pen", 20, 5)
add("Notebook", 60, 10)
add("Bag", 450, 1)
add("Pen", 20, 5)          -> Pen already in cart: quantity becomes 10

Bill: Subtotal 1250 | Discount 125 | Delivery 0 | Total 1125

remove("Bag")              -> Subtotal 800 | Discount 0 | Delivery 0 | Total 800
updateQty("Notebook", 2) -> Subtotal 320 | Discount 0 | Delivery 40 | Total 360
```

Conditions

- Adding a product that is already in the cart increases its quantity instead of adding a second line.
- `updateQty(name, 0)` removes the item.
- Product names are matched ignoring case ("pen" is the same as "Pen").
- All amounts are whole rupees (`int`).

Starter Code

```
class CartItem {
    String name; int price; int qty;
    CartItem(String name, int price, int qty) { /* TODO */ }
    int lineTotal() { /* TODO */ return 0; }
}

public class ShoppingCart {
    private ArrayList<CartItem> items = new ArrayList<>();

    private int indexOf(String name) {
        // TODO: return position of the item, or -1 if not in cart
        return -1;
    }

    public void add(String name, int price, int qty) { /* TODO: use indexOf */ }
    public void remove(String name) { /* TODO */ }
    public void updateQty(String name, int qty) { /* TODO */ }
    public void printBill() { /* TODO: both offer rules */ }
}
```

Step-by-Step Guide (for add)

1. Call `indexOf(name)`.
2. If the result is not -1, fetch that item and increase its `qty`.
3. Otherwise, create a new `CartItem` and add it to the list.
4. Notice that `indexOf` is written once and reused by `add`, `remove` and `updateQty`.

Your Tasks

1. Draw the `ArrayList` after each step of the example (one box per item).
2. Complete both classes, and create a menu-driven program like the one in your earlier assignment.
3. Add `mostExpensiveLine()`, which returns the item with the highest line total.
4. Fill in the complexity of `add`, `remove`, `updateQty` and `printBill` in terms of `n` (the number of cart lines).

Edge Cases to Consider

- Removing an item that is not in the cart
- A negative quantity
- An empty cart (what should the bill show?)
- Subtotal exactly 1000, or amount after discount exactly 500

Common Mistakes

Removing items from an `ArrayList` inside a forward `for` loop. Indexes shift after `remove(i)`, so the next item is skipped.

Writing the discount and delivery checks in the wrong order. Delivery depends on the amount *after* discount.

Think Before Coding

If the store had 10 lakh products in one cart (a wholesale order), which operation would become slow, and why?

Reference

Course portal, Day 2 (`ArrayList` internals). Also read the Java documentation for `ArrayList.remove(int)`.

On the Job

Every e-commerce site (Amazon, Flipkart, Swiggy) has a cart service built on exactly these operations, plus offer rules.

Question 4 - Crash Report: Printing a Stack Trace

Level 4 of 10 · Topic: Stack using an array · Day 2 · Beginner to Intermediate

Scenario

When a Java program crashes, it prints a **stack trace**, the list of methods that were running, the most recent first. You are building a small monitoring tool that reads an application's log of method calls and produces the same kind of report when a CRASH line appears.

Concept Refresher

A stack is Last-In-First-Out (LIFO). `push` adds on top, `pop` removes from the top, and `peek` looks at the top without removing it. When a method starts it is pushed, and when it returns it is popped. This is exactly how the JVM's call stack works.

After ENTER main, ENTER placeOrder, ENTER validateCart:

```
| validateCart | <- top
| placeOrder  |
| main        |
+-----+
```

Example

Log:

```
1: ENTER main
2: ENTER placeOrder
3: ENTER validateCart
4: EXIT validateCart
5: ENTER processPayment
6: ENTER connectBank
7: CRASH
```

Output:

```
Application crashed at line 7. Stack trace:
    at connectBank
    at processPayment
    at placeOrder
    at main
Maximum call depth reached: 4
```

Conditions

- Write your own `MethodStack` using a `String[]` array of capacity 10. Do not use `java.util.Stack`.
- If a push is attempted on a full stack, report `"StackOverflowError: call depth exceeded 10"`.
- An `EXIT x` line where `x` is **not** on top of the stack is an invalid log. Report the line number.
- If the log ends without a CRASH, print `"Program finished normally"`.

Starter Code

```
class MethodStack {
    private String[] data = new String[10];
    private int top = -1;          // -1 means the stack is empty

    boolean push(String m) { /* TODO: return false if full */ return false; }
    String pop()             { /* TODO */ return null; }
    String peek()            { /* TODO */ return null; }
    boolean isEmpty()        { /* TODO */ return true; }
    int size()               { /* TODO */ return 0; }
    void printTrace()        { /* TODO: from top down to index 0 */ }
}

public class CrashReporter {
    public static void main(String[] args) {
        String[] log = { "ENTER main", "ENTER placeOrder", /* ... */ "CRASH" };
        // TODO: read each line, split into command and method name
    }
}
```

Step-by-Step Guide

1. Split each line with `line.split(" ")`. Part 0 is the command, and part 1 (if present) is the method name.
2. ENTER: push the method name. After each push, update `maxDepth` if the size is larger.
3. EXIT: if `peek()` equals the method name, pop it. Otherwise, report an invalid log and stop.
4. CRASH: print the trace from the top down to the bottom, and stop.

Your Tasks

1. Draw the stack after every line of the example.
2. Complete `MethodStack` and test it alone first: push 3 names, pop 1, and print.
3. Complete `CrashReporter` and produce the example output.
4. Create one log that causes a stack overflow, and one with an invalid EXIT. Show both outputs.
5. Give the time complexity of push, pop, peek and printTrace.

Edge Cases to Consider

- EXIT when the stack is empty
- CRASH as the very first line
- Exactly 10 nested calls (allowed), and then an 11th call

Common Mistakes

Using `top = 0` for an empty stack and then pushing at `data[++top]`. This leaves index 0 unused. Printing the trace from index 0 upward, which prints the oldest call first (the reverse of a real stack trace).

Think Before Coding

In Day 4 we saw that infinite recursion causes `StackOverflowError`. Using this problem, explain in your own words what is overflowing.

Reference

Course portal, Day 2 (Stack using array) and Day 4 (call stack and memory). VisuAlgo – Stack: visualgo.net/en/list (select Stack).

On the Job

Crash reporters (Firebase Crashlytics, Sentry), debuggers and log analysers rebuild call stacks this way.

Question 5 - Office Printer Queue

Level 5 of 10 · Topic: Queue implemented with a linked list · Day 2, 3 · Intermediate

Scenario

A shared office printer receives print jobs from many employees. Jobs are printed in the order they arrive (first come, first served). The printer prints **one page every 6 seconds**. Employees can **cancel** a job that is still waiting. The admin wants to see, for each job, when it will finish.

Concept Refresher

A queue is First-In-First-Out (FIFO). Here you implement it with a linked list: each job is a node, `front` points to the first job and `rear` points to the last. Adding at the rear and removing from the front are both $O(1)$, and unlike an array the queue never becomes “full”.

```
front                                rear
|                                   |
[J1 Ravi 25] -> [J2 Asha 10] -> [J3 Karan 5] -> null
```

Example

All jobs arrive at time 0: J1 (Ravi, 25 pages), J2 (Asha, 10 pages), J3 (Karan, 5 pages)

Schedule without cancellation:

```
J1 finishes at 150 s
J2 finishes at 210 s
J3 finishes at 240 s
```

After cancel("J2"):

```
J1 finishes at 150 s
J3 finishes at 180 s
```

Conditions

- Implement `PrintQueue` with your own `Node` class. Do not use `java.util` queues or lists.
- Operations: `enqueue(job)`, `dequeue()`, `peek()`, `isEmpty()`, `size()`, `cancel(jobId)` and `printSchedule()`.
- `printSchedule()` must not remove any jobs from the queue.

Starter Code

```
class Job { String id, owner; int pages; /* TODO: constructor */ }
class Node { Job job; Node next; Node(Job j) { job = j; } }

public class PrintQueue {
    private Node front, rear;
    private int size;

    public void enqueue(Job j) { /* TODO: if empty, front = rear = new node */ }
    public Job dequeue() {
        // TODO: remove from front; if queue becomes empty, set rear = null too
        return null;
    }
    public boolean cancel(String id) {
        // TODO: find the node BEFORE the one to delete, then unlink it
        // TODO: special cases - deleting front, deleting rear
        return false;
    }
    public void printSchedule() { /* TODO: walk from front, keep a running time */ }
}
```

Step-by-Step Guide (for printSchedule)

1. Start with `time = 0` and `temp = front`.
2. While `temp != null`: add `temp.job.pages * 6` to `time`, print the job id with `time`, and move `temp` to `temp.next`.
3. Because you only move `temp`, the queue itself is unchanged.

Your Tasks

1. Draw the linked list and the `front` and `rear` pointers after each enqueue, dequeue and cancel.
2. Complete the class and reproduce both schedules in the example.
3. Test `cancel` on the first job, the last job and a middle job. After each, print `front` and `rear` to confirm they are correct.
4. Fill in the complexity of each operation. Why is `cancel` slower than `dequeue`?

Edge Cases to Consider

- Dequeue from an empty queue
- Cancelling the only job (both `front` and `rear` must become null)
- Cancelling the last job (`rear` must move back)
- Cancelling a job id that does not exist

Common Mistakes

Forgetting to update `rear` when the last node is removed. The next enqueue then attaches to a deleted node.

Moving `front` itself to traverse the list in `printSchedule`, which loses the whole queue.

Think Before Coding

A pure queue does not allow removal from the middle. Why is `cancel` still a reasonable feature here, and what does it cost?

Reference

Course portal, Day 2 (Queue) and Day 3 (stack and queue on a linked list). VisuAlgo – Linked List: visualgo.net/en/list

On the Job

Print spoolers, email outboxes, background job queues (for example, sending OTP SMS) and order processing pipelines.

Question 6 - Train Coach Arrangement

Level 6 of 10 · Topic: Singly linked list operations · Day 3 · Intermediate

Scenario

A railway yard software manages how coaches are attached to a train. The engine is always first. Coaches can be attached at the end, inserted after a specific coach (for example, a pantry car), or detached for maintenance. Station display boards also need the position of each coach from the engine.

Concept Refresher

In a singly linked list, each node knows only its **next** node. To insert or delete in the middle, you must stop at the node **just before** the position, because that is the node whose **next** must change.

Insert PC after S2:

Before: ENGINE -> S1 -> S2 -> S3 -> GEN1

Step 1: newNode.next = S2.next (PC now points to S3)

Step 2: S2.next = newNode (S2 now points to PC)

After: ENGINE -> S1 -> S2 -> PC -> S3 -> GEN1

Example

```
attach("S1", SLEEPER)   attach("S2", SLEEPER)
attach("S3", SLEEPER)   attach("GEN1", GENERAL)
insertAfter("S2", "PC", PANTRY)
detach("S3")
```

```
display()           -> ENGINE -> S1 -> S2 -> PC -> GEN1
countType(SLEEPER) -> 2
position("PC")      -> Coach PC is at position 3 from the engine
totalCoaches()      -> 4 (engine not counted)
```

Conditions

- Use your own `Coach` node class. The engine is the head node, and it can never be detached.
- Keep a `tail` reference so that `attach` is $O(1)$. Every method that can change the last coach must keep `tail` correct.
- Coach ids are unique. Reject a duplicate id with a message.

Starter Code

```
class Coach { String id; String type; Coach next; /* TODO: constructor */ }

public class Train {
    private Coach head = new Coach("ENGINE", "ENGINE");
    private Coach tail = head;
    private int count = 0;    // coaches, excluding engine

    public void attach(String id, String type) { /* TODO */ }
    public boolean insertAfter(String afterId, String id, String type) {
        /* TODO */ return false;
    }
    public boolean detach(String id) { /* TODO: find previous node */ return false; }
    public int position(String id) { /* TODO: 1 = first coach */ return -1; }
    public int countType(String type) { /* TODO */ return 0; }
    public void display() { /* TODO */ }
}
```

Step-by-Step Guide (for detach)

1. If the id is "ENGINE", refuse.
2. Start with `prev = head`. While `prev.next != null` and `prev.next.id` is not the id, move `prev` forward.
3. If `prev.next` is null, the coach was not found.
4. Otherwise, if `prev.next == tail`, set `tail = prev`. Then set `prev.next = prev.next.next` and decrease count.

Your Tasks

1. Draw the list after every operation of the example, using boxes and arrows.
2. Complete the class and match the example output.
3. Add `reverseCoaches()`, which reverses the order of the coaches while keeping the engine first. (Hint: reverse the list that starts at `head.next`, then attach it back to the engine.)
4. Fill in the complexity of each method.

Edge Cases to Consider

- Detaching the last coach (does `tail` move?)
- `insertAfter` the last coach (does `tail` move?)
- Detaching from a train with no coaches
- A coach id that does not exist

Common Mistakes

Doing step 2 before step 1 in the insert diagram. Once `S2.next` changes, you lose the reference to `S3`.

Stopping at the coach to delete instead of the coach before it. In a singly linked list you can't go back.

Think Before Coding

Detaching the **last** coach still needs a full walk, even though you have a `tail` reference. Why? Which structure (next question) fixes this?

Reference

Course portal, Day 2 (singly linked list) and Day 3 (linked lists). VisuAlgo – Linked List: visualgo.net/en/list

On the Job

Linked lists appear wherever items are frequently inserted in the middle: undo histories, memory blocks, and dependency chains.

Question 7 - Hospital Waiting List

Level 7 of 10 · Topic: Doubly linked list · Day 3 · Intermediate

Scenario

A hospital OPD (outpatient department) maintains a waiting list shown on a TV screen. Normal patients join at the **end**. Emergency patients are placed at the **front**. The doctor calls the patient at the front, and a patient may **leave** from anywhere in the line. The screen shows the list from front to back, and the receptionist's screen shows it from back to front.

Concept Refresher

In a doubly linked list each node has **prev** and **next** references. Given a node, you can remove it without searching for the previous node, and you can walk the list in both directions.

```
head                                tail
|                                  |
null <- [Vik] <-> [Riya] <-> [Uma] -> null

Removing a node X:  X.prev.next = X.next
                   X.next.prev = X.prev    (check for null at the ends!)
```

Example

```
addNormal("Riya")   addNormal("Sam")   addEmergency("Tom")   addNormal("Uma")
                                list: Tom, Riya, Sam, Uma
callNext()         -> "Tom, please go to the doctor"
leave("Sam")        -> list: Riya, Uma
addEmergency("Vik")

display()           -> Vik, Riya, Uma
displayReverse()    -> Uma, Riya, Vik
waiting()           -> 3
```

Conditions

- Use your own `Patient` node class with `prev` and `next`. Keep both `head` and `tail`.
- `displayReverse()` must walk from `tail` using `prev` links. This is how you prove your `prev` links are correct.
- Patient names are unique in the list.

Starter Code

```
class Patient { String name; Patient prev, next; Patient(String n) { name = n; } }

public class WaitingList {
    private Patient head, tail;
    private int size;

    public void addNormal(String name)    { /* TODO: insert at tail */ }
    public void addEmergency(String name) { /* TODO: insert at head */ }
    public String callNext()              { /* TODO: remove head */ return null; }
    public boolean leave(String name)     { /* TODO: find, unlink */ return false; }
    public void display()                  { /* TODO: head -> tail */ }
    public void displayReverse()           { /* TODO: tail -> head */ }
}
```

Step-by-Step Guide (for leave)

1. Walk from head to find the node with that name. If it's not found, return false.
2. If the node has a prev, set `node.prev.next = node.next`. Otherwise it was the head, so set `head = node.next`.
3. If the node has a next, set `node.next.prev = node.prev`. Otherwise it was the tail, so set `tail = node.prev`.
4. Decrease `size`. Write a small helper `unlink(Patient p)` and reuse it in `callNext()` as well.

Your Tasks

1. Draw the list with both arrows after each step of the example.
2. Complete the class. After every operation, print both `display()` and `displayReverse()`. They must always be mirror images.
3. **Fairness fix:** With the current rule, a second emergency patient is served *before* the first one. Change `addEmergency` so that emergencies are served in their order of arrival, but still before all normal patients.
4. Compare this structure with the singly linked list from Question 6. Which operations became easier?

Edge Cases to Consider

- `callNext()` on an empty list
- The only patient leaves (both `head` and `tail` become null)
- The first or last patient leaves
- Emergency added to an empty list

Common Mistakes

Updating `next` links but forgetting `prev` links. `display()` looks correct, but `displayReverse()` shows the bug.

Not handling the case where the removed node is the head or the tail, which causes a `NullPointerException`.

Think Before Coding

For the fairness fix, how do you find where the last emergency patient is? Could an extra reference (like `lastEmergency`) make it $O(1)$?

Reference

Course portal, Day 3 (doubly linked list, `java.util.LinkedList` internals). VisuAlgo – Doubly Linked List: visualgo.net/en/list (select DLL).

On the Job

Hospital and bank token displays, browser tabs, and the internal implementation of `java.util.LinkedList`.

Question 8 - Banking Utilities Using Recursion

Level 8 of 10 · Topic: Recursion basics · Day 4 · Intermediate

Scenario

The KYC and accounts team of a bank needs a small utility class. Your team lead asks you to write every method **recursively** (no loops) so that the team can review your understanding of recursion.

Method	Purpose	Example
<code>countDigits(long n)</code>	Validate account number length	4096013 -> 7
<code>sumDigits(long n)</code>	Used in check-digit calculation	4096013 -> 23
<code>digitalRoot(long n)</code>	Keep summing digits until one digit is left	4096013 -> 23 -> 5
<code>mask(String acc)</code>	Hide all but the last 4 characters on statements	"123456789012" -> "XXXXXXXXX9012"
<code>amount(double p, double r, int years)</code>	Compound interest: amount after n years	(10000, 10, 3) -> 13310.00

Concept Refresher

Every recursive method has two parts. The **base case** answers the smallest input directly, without recursion. The **recursive case** solves a smaller version of the same problem and uses that answer. For digits, the smaller problem is usually $n / 10$ (the number without its last digit), and the last digit is $n \% 10$.

Example: Tracing `countDigits(4096)`

```
countDigits(4096) = 1 + countDigits(409)
  countDigits(409) = 1 + countDigits(40)
    countDigits(40) = 1 + countDigits(4)
      countDigits(4) = 1                    <- base case, no further call
    countDigits(40) = 1 + 1 = 2            <- returning back up
  countDigits(409) = 1 + 2 = 3
countDigits(4096) = 1 + 3 = 4
```

Conditions

- No `for` or `while` loops anywhere in the class.
- Do not convert numbers to `String` for the digit methods.
- Print `amount` with two decimal places (the raw `double` result prints as 13310.0000000000004. Find out why).

Starter Code

```
public class BankUtils {
    static int countDigits(long n) {
        if (/* TODO: base case */) return 1;
        return /* TODO */;
    }
    static int sumDigits(long n)      { /* TODO */ return 0; }
    static int digitalRoot(long n)    { /* TODO: use sumDigits */ return 0; }
    static String mask(String acc)    { /* TODO: think about the length */ return ""; }
    static double amount(double p, double r, int years) {
        // amount after 0 years = p
        // amount after n years = amount after (n-1) years * (1 + r/100)
        return 0.0;
    }
}
```

Step-by-Step Guide (for mask)

1. Base case: if the string length is 4 or less, return it unchanged.
2. Recursive case: return "X" + mask(acc.substring(1)).
3. Trace it for "123456": X + mask("23456") = X + X + mask("3456") = "XX3456".

Your Tasks

1. For each method, write the base case and the recursive case in plain English **before** writing code.
2. Trace sumDigits(4096013) and digitalRoot(4096013) in the same style as the example above.
3. Complete all the methods and test them with the values in the table.
4. For each method, state the time complexity and the maximum depth of the call stack.

Edge Cases to Consider

- n = 0 (how many digits does 0 have?)
- A single-digit number
- An account string shorter than 4 characters
- years = 0
- A negative number (decide: reject it, or use its absolute value)

Common Mistakes

A missing or wrong base case, which causes `StackOverflowError`.

Calling the method on the same value instead of a smaller one (for example `countDigits(n)` inside `countDigits`).

Think Before Coding

`digitalRoot` calls `sumDigits`, and also calls itself. Draw the calls for 4096013. How many times is each method called?

Reference

Course portal, Day 4 (base condition, call stack and memory). Revise the call-stack diagrams from class.

On the Job

Check-digit validation (account numbers, IFSC, credit cards), data masking for privacy rules, and interest calculation.

Question 9 - Payment Kiosk Coin Combinations

Level 9 of 10 · Topic: Recursion tree, repeated work · Day 4 · Intermediate

Scenario

A parking payment kiosk accepts only **Rs. 1 and Rs. 2 coins**. The product team wants to test the kiosk with *every* possible order in which a customer could insert coins to pay an amount. For example, paying Rs. 3 could be 1+1+1, 1+2 or 2+1. The order matters, because the machine records each coin.

You must (a) print every sequence, and (b) count the sequences for larger amounts, where printing them all is impossible.

Concept Refresher

At each step the customer has a choice: insert a Rs. 1 coin or a Rs. 2 coin. Each choice leaves a **smaller amount** to pay, which is the same problem again. Drawing every choice as a branch gives a **recursion tree**.

Recursion tree for amount 3:

```

              pay(3)
            +1 /   \ +2
          pay(2)   pay(1)
        +1 /   \ +2   | +1
      pay(1) pay(0)* pay(0)*
        | +1
      pay(0)*
Sequences: 1 1 1 | 1 2 | 2 1
```

* = paid in full, one valid sequence

Example

```
Amount = 4
1 1 1 1
1 1 2
1 2 1
2 1 1
2 2
Total ways = 5

countWays(5) = 8      countWays(10) = 89      countWays(30) = 1346269
```

Conditions

- Both `printWays` and `countWays` must be recursive.
- `printWays` passes the “coins so far” down as a parameter (a `String` is fine).
- Add a static counter that counts how many times `countWays` is called.

Starter Code

```
public class CoinKiosk {
    static long calls = 0;

    static void printWays(int amount, String coinsSoFar) {
        if (amount == 0) { /* TODO: print coinsSoFar */ return; }
        // TODO: try a Rs.1 coin
        // TODO: try a Rs.2 coin (only if amount >= 2)
    }

    static long countWays(int amount) {
        calls++;
        // TODO: base cases for amount 0 and amount 1
        // TODO: ways(amount) = ways(amount - 1) + ways(amount - 2)
        return 0;
    }
}
```

Your Tasks

1. Draw the recursion tree for amount 4 and check that it has exactly 5 leaves marked “paid”.
2. Complete `printWays` and `countWays` and match the example.
3. Print `calls` for amounts 10, 20, 30 and 40. Record the results and the time taken in a table. What do you notice?
4. In your tree for amount 4, circle every place where the **same** amount is solved more than once.
5. **Improve it:** Store each answer in a `long[]` *memo* array the first time it is computed, and reuse it later. Count the calls again for amount 40. (We will study this technique, *memoization*, formally on Day 8.)
6. **Extension:** The kiosk now also accepts a Rs. 5 coin. Update `countWays`. Check that amount 5 now gives 9 ways.

Edge Cases to Consider

- Amount 0 (is there one way, or zero?)
- Amount 1
- A negative amount reached inside the recursion

Common Mistakes

Missing the `amount >= 2` check, which recurses into negative amounts forever.
Using `int` for the count. It overflows for amounts above about 45. Use `long`.

Think Before Coding

The numbers 1, 2, 3, 5, 8, 13 ... look familiar. Which famous sequence is this, and why does this problem produce it?

Reference

Course portal, Day 4 (recursion and complexity of recursive functions). Visual tool: search “recursion tree visualizer” to see the tree for small amounts.

On the Job

Testing tools that generate every input combination, route counting in maps, and pricing-plan combinations.

Question 10 – Library Book Catalogue

Level 10 of 10 · Topic: Binary search tree, traversals · Day 5 · Intermediate

Scenario

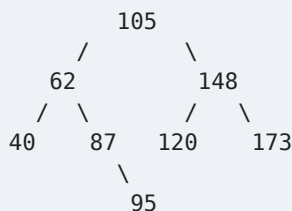
A college library is moving from a register to software. Each book has a unique numeric **Book ID** and a title. The librarian needs to add books, search for a book quickly by ID, print the full catalogue in ID order, and see the smallest and largest IDs. The development team decides to store the catalogue in a **binary search tree (BST)** keyed by Book ID.

Concept Refresher

In a BST, every node's left subtree has smaller keys and its right subtree has larger keys. To search, compare with the current node and go left or right, so half of the remaining tree is ignored at each step (when the tree is balanced). An **inorder** traversal (Left, Node, Right) prints the keys in sorted order.

Example

Insert Book IDs in this order: 105, 62, 148, 40, 87, 120, 173, 95



Inorder (catalogue) : 40 62 87 95 105 120 148 173
Preorder : 105 62 40 87 95 148 120 173
Postorder : 40 95 87 62 120 173 148 105
Level order : 105 | 62 148 | 40 87 120 173 | 95

search(95) -> Found. Path: 105 -> 62 -> 87 -> 95
search(100) -> Not found. Path: 105 -> 62 -> 87 -> 95 -> (no right child)
min = 40, max = 173, count = 8, height = 3 (counting edges)

Conditions

- Use your own `BookNode` class with `id`, `title`, `left` and `right`.
- Inserting an ID that already exists must print “Duplicate Book ID” and not change the tree.
- `search` must print the path it followed.
- For level order, use the queue you wrote in Question 5 (with `BookNode` instead of `Job`).

Starter Code

```
class BookNode { int id; String title; BookNode left, right; /* TODO: constructor */ }

public class Catalogue {
    private BookNode root;

    public void insert(int id, String title) { root = insert(root, id, title); }
    private BookNode insert(BookNode node, int id, String title) {
        // TODO: if node is null, create and return a new node
        // TODO: go left or right; duplicates are rejected
        return node;
    }
    public BookNode search(int id) {
        // TODO: iterative is fine; print the path followed
        return null;
    }
    public void inorder(BookNode n) { /* TODO */ }
    public void preorder(BookNode n) { /* TODO */ }
    public void postorder(BookNode n) { /* TODO */ }
    public int count(BookNode n) { /* TODO */ return 0; }
    public int height(BookNode n) { /* TODO: return -1 for null */ return -1; }
    public int min() { /* TODO: keep going left */ return -1; }
    public int max() { /* TODO: keep going right */ return -1; }
}
```

Step-by-Step Guide (for the traversals)

- **Inorder:** if the node is null, return. Otherwise visit the left subtree, then print the node, then visit the right subtree.
- **Preorder:** print the node first, then the left subtree, then the right subtree.
- **Postorder:** the left subtree, then the right subtree, then print the node.
- Only the position of the print line changes. The rest of the code is identical.

Your Tasks

1. Draw the BST yourself for the insert order above, without looking at the example tree, and then compare.
2. Complete all the methods and reproduce the example output.
3. Insert the same 8 IDs in **sorted order** (40, 62, 87, ..., 173) into a new tree. Draw it and print its height. What has the tree become?
4. Fill in this table, with n = number of books and h = height of the tree:

Operation	Balanced tree	Tree built from sorted IDs
search	?	?
insert	?	?
inorder (print all)	?	?
min / max	?	?

Edge Cases to Consider

- An empty catalogue (min, max and search must not crash)
- A duplicate Book ID
- A tree with a single book
- Searching for the smallest and the largest ID

Common Mistakes

Writing `insert(node.left, id, title)` without assigning the result back to `node.left`, so new books are never attached.

Mixing up the height conventions (edges vs nodes). State which one you use, and keep it consistent.

Think Before Coding

Why does an inorder traversal of a BST always come out sorted? Explain it in two lines, using the BST rule.

Reference

Course portal, Day 5 (terminology, traversals, BST operations). VisuAlgo – BST: visualgo.net/en/bst (insert the same IDs there and compare with your drawing).

On the Job

Database indexes, file system directories and in-memory lookups all use search trees. Day 6 covers how they stay balanced.